

Лабораторная работа № 4. Линейная алгебра.

Абакумова О. М.

Российский университет дружбы народов, Москва, Россия

Информация

- Абакумова Олеся
Максимовна
- Студентка
- Российский университет
дружбы народов
- 1132220832@pfur.ru
- <https://github.com/omabakumova>



Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

Задание

1. Выполнить примеры, приведенные в лабораторной работе.
2. Выполнить задания для самостоятельного выполнения

Выполнение лабораторной работы

Поэлементные операции над многомерными массивами

```
Поэлементные операции над многомерными массивами

(1): # Массив 4x3 со случайными целыми числами (от 1 до 20):
    a = randi(1,20,[4,3])

(2): 4x3 Matrix (int64):
     4 10 6
     8 5 5
     3 14 12
    16 18 17

(3): # Поэлементная сумма:
    sum(a)

(4): 1x3 Matrix (int64):
     31 56 40

(5): # Поэлементная сумма по строкам:
    sum(a,dims=2)

(6): 4x1 Matrix (int64):
     29
     18
     29
     51

(7): # Поэлементное произведение:
    prod(a)

(8): 22504366888

(9): # Поэлементное произведение по столбцам:
    prod(a,dims=1)

(10): 1x3 Matrix (int64):
     1536 2384 6120

(11): # Поэлементное произведение по строкам:
    prod(a,dims=2)

(12): 4x1 Matrix (int64):
     456
     288
     504
     4896

(13): using Statistics

(14): # Вычисление среднего значения массива:
    mean(a)

(15): 10.583333333333334

(16): # Среднее по столбцам:
    mean(a,dims=1)

(17): 1x3 Matrix (Float64):
     7.75 14.0 10.0

(18): # Среднее по строкам:
    mean(a,dims=2)

(19): 4x1 Matrix (Float64):
     9.555555555555556
     6.0
     9.555555555555556
    17.0
```

Рис. 1: Примеры поэлементных операций

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) **LinearAlgebra**.

Транспонирование, след, ранг, определитель и инверсия матрицы

```
using LinearAlgebra

# Массив 4x4 с случайными целыми числами (от 1 до 20):
b = rand{Int64}(4,4)

4x4 Matrix{Int64}:
 5  9  5 12
 8  2 16 11
 6  7 18 11
14 18 18  9

# Транспонирование:
ttranspose(b)

4x4 Transpose{Matrix{Int64}} with eltype Int64:
 5  8  6 14
 9  2  7 18
 5 18 18 18
12 11 11  9

# След матрицы (сумма диагональных элементов):
tr(b)

26

# Извлечение диагональных элементов как массив:
diag(b)

4-element Vector{Int64}:
 5
 2
18
 9

# Ранг матрицы:
rank(b)

4

# Инверсия матрицы (определение обратной матрицы):
inv(b)

4x4 Matrix{Float64}:
 0.0239863  0.383871  -0.22537  0.0779696
 0.0895854  -0.128052  0.097052  0.0180263
 -0.17327  -0.012283  0.279086  -0.0311385
 0.159425  0.0620725  -0.081459  -0.0901183

# Определитель матрицы:
det(b)

4199.000000000001

# Псевдообратная функция для прямоугольных матриц:
pinv(a)

3x4 Matrix{Float64}:
 0.0114492  0.0649661  -0.097037  0.0451513
 0.080723  -0.0613962  -0.030552  -0.0345252
 -0.00985  -0.0371761  0.117063  0.0290437
```

Рис. 2: Различные операции над матрицами

Евклидова норма:

$$\|\vec{X}\|_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$$

p-норма:

$$\|\vec{A}\|_p = \left(\sum_{i=1}^n |a_i|^p \right)^{1/p}$$

Вычисление нормы векторов и матриц, повороты, вращения

Вычисление нормы векторов и матриц, повороты, вращения	
1201	<pre># Создание вектора A: A = [2, 4, -5]</pre>
1202	<pre># элемент Vector(Det54): 2 4 -5</pre>
1203	<pre># Вычисление евклидовой нормы: norm(A)</pre>
1204	<pre>6.782205115693989</pre>
1205	<pre># Вычисление p-нормы: p = 1 norm(A,p)</pre>
1206	<pre>11.6</pre>
1207	<pre># Расстояние между двумя векторами X и Y: X = [2, 4, -5] Y = [1,-3,3] norm(X,Y)</pre>
1208	<pre>9.406117089505138</pre>
1209	<pre># Повернуть по формуле отклонения: rotY(X)(X,Y,"Y")</pre>
1210	<pre>9.406117089505138</pre>
1211	<pre># Раск между двумя векторами: abs((X.transpose(X*Y)-Y.transpose(X*Y))/2)</pre>
1212	<pre>2.486487769465052</pre>
1213	<pre># Создание матрицы: A = [1 -4 2 ; -1 2 3 ; -2 1 8]</pre>
1214	<pre>3x3 Matrix(Det54): 1 -4 2 -1 2 3 -2 1 8</pre>
1215	<pre># Вычисление Евклидовой нормы: norm(A)</pre>
1216	<pre>7.147682841762368</pre>
1217	<pre># Вычисление p-нормы: p=1 norm(A,p)</pre>
1218	<pre>8.6</pre>
1219	<pre># Поворот на 90 градусов: rot90(A)</pre>
1220	<pre>3x3 Matrix(Det54): 0 1 -1 0 2 -1 2 -4 3</pre>
1221	<pre># Поворотоминимум строк: rotate(A,0,0=1)</pre>
1222	<pre>3x3 Matrix(Det54): 2 1 8 -1 2 3 0 -4 2</pre>
1223	<pre># Поворотоминимум столбцов: rotate(A,0,0=2)</pre>
1224	<pre>3x3 Matrix(Det54): 2 -4 3 0 2 -1 0 1 -2</pre>

Рис. 3: Вычисления нормы векторов и матриц, повороты, вращения

Матричное умножение, единичная матрица, скалярное произведение

```
Матричное умножение, единичная матрица, скалярное произведение

[34]: # Матрица 2x3 со случайными целыми значениями от 1 до 10:
      A = rand(1:10,(2,3))

[34]: 2x3 Matrix{Int64}:
      5  8  7
      2  9  7

[35]: # Матрица 3x4 со случайными целыми значениями от 1 до 10:
      B = rand(1:10,(3,4))

[35]: 3x4 Matrix{Int64}:
      10  9  3  2
      8  8  5  6
      3  2 10  8

[36]: # Произведение матриц A и B:
      A*B

[36]: 2x4 Matrix{Int64}:
      135 123 125 114
      113 104 121 114

[37]: # Единичная матрица 3x3:
      Matrix{Int}(I, 3, 3)

[37]: 3x3 Matrix{Int64}:
      1  0  0
      0  1  0
      0  0  1

[38]: # Скалярное произведение векторов X и Y:
      X = [2, 4, -5]
      Y = [1, -1, 3]
      dot(X,Y)

[38]: -17

[39]: # Тоже скалярное произведение:
      X*Y

[39]: -17
```

Рис. 4: Примеры матричного умножения, единичной матрицы, скалярного произведения

В математике факторизация (или разложение) объекта — его декомпозиция (например, числа, полинома или матрицы) в произведение других объектов или факторов, которые, будучи перемноженными, дают исходный объект.

Рассмотрим несколько примеров. Для работы со специальными матричными структурами потребуется пакет **LinearAlgebra**.

Факторизация. Специальные матричные структуры

```
1391: -37

Факторизация. Специальные матричные структуры

1401: # Задаём квадратичную матрицу 3x3 со случайными значениями:
1402: A = rand(3, 3)

1403: 3x3 Matrix{Float64}:
0.932133  0.375629  0.340507
0.451387  0.525771  0.236173
0.773263  0.346031  0.121879

1404: # Задаём единичный вектор:
1405: u = [1;1;1;0, 2]

1406: 3-element Vector{Float64}:
1.0
1.0
1.0

1407: # Задаём вектор b:
1408: b = A*u

1409: 3-element Vector{Float64}:
1.6062802255679325
1.6127094676242095
1.3882732836422582

1410: # Проверим, насколько правильно получаем с помощью функции 1
# (убедимся, что u - единичный вектор)
1411: u/b

1412: 3-element Vector{Float64}:
0.6000000000000001
1.0000000000000002
1.0000000000000001

1413: # L-D-факторизация:
1414: LU = lu(A)

1415: LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3x3 Matrix{Float64}:
1.0  0.0  0.0
0.515499  1.0  0.0
0.827283  0.193748  1.0
D factor:
3x3 Matrix{Float64}:
0.932133  0.375629  0.340507
0.0  0.179832  -0.0823853
0.0  0.0  -0.14894

1416: # Матрица перестановки:
1417: PU = P

1418: 3x3 Matrix{Float64}:
1.0  0.0  0.0
0.0  1.0  0.0
0.0  0.0  1.0

1419: # Вектор перестановки:
1420: PU.p

1421: 3-element Vector{Int64}:
1
2
3

1422: # Матрица L:
1423: LU.L

1424: 3x3 Matrix{Float64}:
1.0  0.0  0.0
0.515499  1.0  0.0
0.827283  0.193748  1.0

1425: # Матрица D:
1426: LU.D

1427: 3x3 Matrix{Float64}:
0.932133  0.375629  0.340507
0.0  0.179832  -0.0823853
0.0  0.0  -0.14894
```

Рис. 5: Примеры работы со специальными матричными структурами

Исходная система уравнений $Ax = b$ может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации.

Факторизация. Специальные матричные структуры

```
[10]: # Решение СДУ через матрицу A:  
A1D  
  
[11]: 3-element Vector{Float64}:  
 0.9999999999999999  
 1.0000000000000000  
 1.0000000000000001  
  
[12]: # Решение СДУ через объект факторизации:  
A1a1D  
  
[13]: 3-element Vector{Float64}:  
 0.9999999999999999  
 1.0000000000000000  
 1.0000000000000001  
  
[14]: # Детерминант матрицы A:  
det(A)  
  
[15]: -0.823494128586286225  
  
[16]: # Детерминант матрицы A через объект факторизации:  
det(A1a1D)  
  
[17]: -0.823494128586286225  
  
[18]: # QR-факторизация:  
Aqr = qr(A)  
  
[19]: LinearAlgebra.QRCompactWY{TFloat64, Matrix{TFloat64}, Matrix{TFloat64}}  
Q factor: 3x3 LinearAlgebra.QRCompactWY{TFloat64, Matrix{TFloat64}, Matrix{TFloat64}}  
R factor:  
3x3 Matrix{TFloat64}:  
-1.48942 -0.718852 -0.424132  
0.0 -0.139889 0.833637  
0.0 0.0 -0.126768  
  
[20]: # Матрица Q:  
Aqr.Q  
  
[21]: 3x3 LinearAlgebra.QRCompactWY{TFloat64, Matrix{TFloat64}, Matrix{TFloat64}}  
  
[22]: # Матрица R:  
Aqr.R  
  
[23]: 3x3 Matrix{TFloat64}:  
-1.48942 -0.718852 -0.424132  
0.0 -0.139889 0.833637  
0.0 0.0 -0.126768  
  
[24]: # Проверка, что матрица Q - ортогональная:  
Aqr.Q' * Aqr.Q  
  
[25]: 3x3 Matrix{TFloat64}:  
1.0 0.0 -1.18229e-16  
0.0 1.0 0.0  
0.0 0.0 1.0  
  
[26]: # Симметричная матрица A:  
A1sym = A + A'  
  
[27]: 3x3 Matrix{TFloat64}:  
1.98427 1.228 1.13937  
1.228 1.98484 0.582784  
1.13937 0.582784 0.262158  
  
[28]: # Собственные значения симметричной матрицы:  
Aqr1eig = eigen(A1sym)  
  
[29]: Eigen{TFloat64, TFloat64, Matrix{TFloat64}, Vector{TFloat64}}  
values:  
3-element Vector{Float64}:  
-0.331882982418889  
0.2174672479818952  
0.275683729638229  
vectors:  
3x3 Matrix{TFloat64}:  
-0.483761 0.439524 -0.782789  
0.8638746 -0.829962 -0.522426  
0.872924 0.386294 -0.384483  
  
[30]: # Собственные значения:  
Aqr1eig.values  
  
[31]: 3-element Vector{Float64}:  
-0.331882982418889  
0.2174672479818952  
0.275683729638229
```

Факторизация. Специальные матричные структуры

```
[61]: #Специальные матрицы:
      KspMatr(Ksp)

[62]: 3x3 Matrix(Float64):
      0.482761  0.430624 -0.762766
      0.6620746 -0.820962  0.522426
      0.872924  0.390294 -0.384483

[63]: # Проверим, что получится единичная матрица:
      Inv(KspMatr(Ksp))

[64]: 3x3 Matrix(Float64):
      1.0      1.55431e-15  3.33067e-16
      -1.33227e-15  1.0      0.66134e-16
      1.33227e-15  -3.22804e-16  1.0

[65]: # Матрица 1000 x 1000:
      n = 1000
      A = randn(n,n)

[66]: 1000x1000 Matrix{Float64}:
      0.828448  1.26829  0.356836  ... -2.11284  -1.45814  -1.48844
      -0.180574  1.58535  0.117792  ... -0.365828  -0.287565  0.785482
      -1.06577  1.32781  0.345321  ... -1.08543  1.08595  -0.088055
      -1.65224  -0.898426  0.665208  ... -1.81495  -0.288483  -1.7884
      -0.624674  0.348538  0.616489  ... 0.854665  1.13514  -1.26533
      -1.47903  -0.85529  -0.821392  ... -1.11195  -0.8223531  1.89988
      0.876588  1.08955  -1.15135  ... -0.93542  0.585702  1.14
      -0.299825  0.851777  -0.878415  ... 0.658225  0.188935  1.25651
      -0.246869  1.85481  -0.820261  ... -2.18376  -0.819967  0.6847703
      -0.6277812  0.589627  -1.93684  ... -0.387882  -0.328959  -0.82485
      -2.03577  0.830296  -0.682774  ... -0.247238  -0.254913  -1.68853
      0.367151  0.867827  -0.511461  ... 0.762637  0.837263  0.890375
      1.38547  -0.798487  0.654746  ... 0.8281158  0.651108  -0.548175
      ...
      -0.383486  -0.1381  0.343171  ... -0.818859  0.136223  -0.21117
      1.87888  1.25558  1.72787  ... 1.36745  0.178286  -1.68882
      -0.85328  0.920428  -1.45981  ... -0.412953  0.548842  -0.488124
      -0.408461  -1.04535  -0.855986  ... -1.72991  -0.548505  -0.389054
      -0.391725  -0.350184  1.14246  ... 1.95423  1.33886  -0.115886
      1.8462  0.586523  -0.787431  ... 1.03551  1.16417  -0.789762
      -0.664224  0.868611  2.18978  ... 0.833937  -0.245388  1.1205
      -1.13972  0.129874  -0.379284  ... -0.833525  0.112267  -1.74355
      -1.23381  -1.77842  -0.611832  ... 0.388125  -1.32341  -0.878754
      -0.127265  0.28267  0.648825  ... -0.538278  -0.128978  0.786378
      0.675187  0.488414  0.8889462  ... 1.25206  -0.45730  1.53778
      -0.427728  0.12144  1.85155  ... 0.8339976  -2.58424  0.545735

[67]: # Симметричные матрицы:
      Akm = A + A'

[68]: 1000x1000 Matrix{Float64}:
      -1.6289  1.87971  -1.43021  -2.71769  ... -0.714953  -1.88827
      1.87971  3.1227  1.43021  0.822855  ... 0.288989  0.882322
      -1.43021  1.43051  0.898443  1.8782  ... 0.87888  0.242796
      -2.71769  0.822855  1.8782  2.44286  ... -0.834857  -2.83436
      0.882772  0.894587  0.712958  -0.458466  ... 2.23528  -2.1294
      -1.84611  0.723861  -2.62618  -2.64305  ... -0.8940168  1.13355
      0.348388  1.198  1.24884  0.348451  ... 1.65811  0.888884
      -1.44889  0.827187  1.3388  0.879918  ... 1.58354  0.387781
      1.24467  -0.837563  -1.33887  1.46702  ... -0.275263  1.87729
      -0.212787  0.513466  -2.69202  0.26519  ... 0.8326246  -0.107174
      -1.51362  1.48843  1.25712  0.830814  ... -1.57862  1.48848
      1.38865  2.81826  -2.52862  0.866882  ... -0.819512  0.3982
      1.72119  0.512188  1.23859  0.379849  ... 0.883382  -1.38454
      ...
      -0.83845  0.583889  1.32786  2.85386  ... 0.287423  0.388347
      2.78523  1.29827  0.35323  -2.14453  ... 0.942964  -2.7884
      -0.78885  1.86288  -1.35621  -0.188748  ... -2.8395  -0.168381
      -0.686617  -0.862587  1.35311  -3.174  ... 0.675648  0.128573
      -0.75288  0.8788725  2.87522  -0.714804  ... 1.87543  0.8287712
      1.81212  2.11867  0.828888  -0.888955  ... 0.648893  1.81878
      0.485185  2.18711  3.27376  0.767181  ... 0.542081  2.12788
      -1.18565  0.778133  0.582885  -2.4141  ... 0.518686  -0.634844
      -2.23888  -2.9884  0.158889  -1.71485  ... 0.378851  0.843787
      -2.78884  -0.878278  0.4449  -0.588288  ... 1.38884  -0.843787
      -0.774952  0.288889  1.87889  -0.834937  ... -0.514585  -0.8064848
      1.88817  0.862822  0.342286  -2.83436  ... 0.8884848  1.88351

[69]: # Проверим, является ли матрица симметричной:
      isSymmetric(Akm)

[69]: true

[70]: # Дифференциал функции:
      Akm_nobj = copy(Akm)
      Akm_nobj[1,2:2] += 2*eps(1)

[71]: 1.8787134524244016
```

Рис. 7: Примеры работы с матрицами большой размерности и специальной

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом `BenchmarkTools`.

Факторизация. Специальные матричные структуры

[illegible]

Далее рассмотрим примеры работы с разреженными матрицами большой размерности. Использование типов `Tridiagonal` и `SymTridiagonal` для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами.

Факторизация. Специальные матричные структуры

```
[76]: # Оценка эффективности выполнения операции по нахождению
# собственных значений:
@time eigen(A)

636.302 ss (44 allocations: 383.11 MiB)

[76]: 0.23724085702745

[77]: B = Matrix{A}

SubMemoryError()
Stacktrace:
 [1] GenericMemory
   @ ./memory.jl:936 [inlined]
 [2] new_memory
   @ ./memory.jl:936 [inlined]
 [3] Array
   @ ./array.jl:682 [inlined]
 [4] Matrix{Float64}(M::SymTridiagonal{Float64, Vector{Float64}})
   @ LinearAlgebra ./julia/juliaup/julia-1.11.0-dev-linux-gnu/share/julia/stdlib/v1.11/LinearAlgebra/src/Tridiag.jl:136
 [5] (Matrix{M::SymTridiagonal{Float64, Vector{Float64}}})
   @ LinearAlgebra ./julia/juliaup/julia-1.11.0-dev-linux-gnu/share/julia/stdlib/v1.11/LinearAlgebra/src/Tridiag.jl:147
 [6] top-level scope
   @ In[77]:4
```

Рис. 9: Оценка эффективности

Обычный способ добавить поддержку числовой линейной алгебры - это обернуть подпрограммы **BLAS** и **LAPACK**. Собственно, для матриц с элементами `Float32`, `Float64`, `Complex {Float32}` или `Complex {Float64}` разработчики Julia использовали такое же решение. Однако Julia также поддерживает общую линейную алгебру, что позволяет, например, работать с матрицами и векторами рациональных чисел.

Для задания рационального числа используется двойная косая черта: $1//2$
В следующем примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем **BigInt**).

Общая линейная алгебра

```
Общая линейная алгебра

[76]: 1/2
[76]: 1/2
[76]: # Матрица с рациональными элементами
Arational = Matrix(Rational(BigInt))(rand(1:10, 3, 3))/10
[76]: 3x3 Matrix{Rational{BigInt}}:
 2//10  9//10  1//2
 2//5  1//10  1
 4//5  2//5  7//10
[80]: # Единичный вектор:
e = fill{1, 3}
[80]: 3-element Vector{Int64}:
 1
 1
 1
[81]: # Задаём вектор b:
b = Arational*e
[81]: 3-element Vector{Rational{BigInt}}:
 17//10
 13//10
 19//10
[82]: # Решим исходное уравнение используя функцию \
# Губкина, что e - единичный вектор:
Arational\b
[82]: 3-element Vector{Rational{BigInt}}:
 1
 1
 1
[83]: # 13-го элемента:
b[13](Arational)
[83]: 13(Rational{BigInt}, Matrix{Rational{BigInt}}, Vector{Int64})
 1 0 0
 3x3 Matrix{Rational{BigInt}}:
 1/5 1 0
 1/2 2//15 1
 0 Factor:
 2x3 Matrix{Rational{BigInt}}:
 4//5 2//5 7//10
 0 3//4 20//90
 0 0 272//630
```

Рис. 10: Пример решения системы линейных уравнений с рациональными элементами

Задания для самостоятельного выполнения

- 1.1 Задайте вектор $\mathbf{v}$. Умножьте вектор $\mathbf{v}$ скалярно сам на себя и сохраните результат в `dot_v`.
2. Умножьте $\mathbf{v}$ матрично на себя (внешнее произведение), присвоив результат переменной `outer_v`.

Задания для самостоятельного выполнения

Задания для самостоятельного выполнения

```
[04]: # 4.4.2. Произведение векторов
#1
v = [1, 2, 3] # произвольный вектор-столбец
dot_v = dot(v, v) # скалярное произведение (v^T v)
```

[04]: 14

```
[05]: #2
outer_v = v * v^T # v^T — это транспонированный вектор-строка
```

[05]:

```
3x3 Matrix(Int64):
 1 2 3
 2 4 6
 3 6 9
```

Рис. 11: Пункт 1 и пункт 2

3. Решить СЛАУ с двумя неизвестными.

Задания для самостоятельного выполнения

[illegible]

Рис. 12: Решение СЛАУ с двумя неизвестными

4. Решить СЛАУ с тремя неизвестными.

Задания для самостоятельного выполнения

```
1) P с тремя неизвестными

A2b = [1 1 1; 1 -1 -2]
b2b = [2; 3]
println("A = $A2b, b = $b2b")
println("Решить A: $A2b\b2b")
println("Система неоднородная, имеет бесконечно много решений")
println("Свободные переменные: $1/3 - gain(A2b)")

A = [1 1 1; 1 -1 -2], b = [2; 3]
Реш: A: 2
Система однородная, имеет бесконечно много решений
Свободные переменные: 1

A2b = [1 1 1; 2 2 -3; 3 3 3]
b2b = [2; 4; 3]
println("A = $A2b, b = $b2b")
println("Обратить A: $inv(A2b)")
try
    x2b = A2b \ b2b
    println("Решение: x = $round(x2b[1], digits=3), y = $round(x2b[2], digits=3), z = $round(x2b[3], digits=3)")
catch e
    println("Система не имеет единственного решения")
end

A = [1 1 1; 2 2 -3; 3 3 3], b = [2; 4; 3]
Обратить A: -10.0
Решение: x = -8.5, y = 2.5, z = 8.0

A2b = [1 1 1; 1 1 1; 2 2 3]
b2b = [2; 8; 3]
println("A = $A2b, b = $b2b")
println("Реш A: $A2b\b2b, дан расширенный матрицы: $rank(A2b b2b)")
if rank(A2b) == rank(A2b b2b)
    if rank(A2b) == 3
        println("Система имеет единственное решение")
        x2b = A2b \ b2b
        println("Решение: x = $round(x2b[1]), y = $round(x2b[2]), z = $round(x2b[3])")
    else
        println("Система совместна, имеет бесконечно много решений")
        println("Свободные переменные: $1/3 - gain(A2b)")
    end
else
    println("Система несовместна")
end

A = [1 1 1; 1 1 1; 2 2 3], b = [1; 0; 3]
Реш: A: 2, дан расширенный матрицы: 2
Система совместна, имеет бесконечно много решений
Свободные переменные: 1

A2b = [1 1 1; 1 1 1; 2 2 3]
b2b = [1; 0; 8]
println("A = $A2b, b = $b2b")
println("Реш A: $A2b\b2b, дан расширенный матрицы: $rank(A2b b2b)")
if rank(A2b) == rank(A2b b2b)
    if rank(A2b) == 3
        println("Система имеет единственное решение")
        x2b = A2b \ b2b
        println("Решение: x = $round(x2b[1]), y = $round(x2b[2]), z = $round(x2b[3])")
    else
        println("Система совместна, имеет бесконечно много решений")
    end
else
    println("Система несовместна - нет решений")
end

A = [1 1 1; 1 1 1; 2 2 3], b = [1; 0; 8]
Реш: A: 2, дан расширенный матрицы: 3
Система несовместна - нет решений
```

Рис. 13: Решение СЛАУ с тремя неизвестными

5. Приведите матрицы к диагональному виду.
6. Вычислите.

Задания для самостоятельного выполнения

```
01: # 4.4.3. Операции с матрицами
# 1. Приведем приведенные ниже матрицы к диагональному виду
A1a = [1 -2; -2 1]
Diagonal(eigen(A1a).values)

02: 2x2 Diagonal(Float64, Vector{Float64}):
-1.0  '
      - 3.0

03: A2a = [1 -2; -2 3]
Diagonal(eigen(A2a).values)

04: 2x2 Diagonal(Float64, Vector{Float64}):
-0.236068  '
          4.23607

05: A3a = [1 -2 0; -2 1 2; 0 2 0]
Diagonal(eigen(A3a).values)

06: 3x3 Diagonal(Float64, Vector{Float64}):
-2.14134  '
          0.515138  '
          3.6262

07: # 2. Вычислено
A1a^10

08: 2x2 Matrix{Int64}:
29525  -29524
-29524  29525

09: B = [5 -2; -2 5]
sqrt(B)

10: 2x2 Matrix{Float64}:
2.1889  -0.45685
-0.45685  2.1889

11: A1a^(1/3)

12: 2x2 Symmetric{ComplexF64, Matrix{ComplexF64}}:
0.971125+0.433013im  -0.471125+0.433013im
-0.471125+0.433013im  0.971125+0.433013im

13: K = [1 2; 2 3]
sqrt(K)

14: 2x2 Matrix{ComplexF64}:
0.568844+0.351375im  0.828442-0.217287im
0.928442-0.217287im  1.48931+0.134291im

15: Q = [140  97  74 160 131;
        97 186  89 131  36;
        74  89 152 144  71;
        160 131 144  54 142;
        131  36  71 142  36]

16: 5x5 Matrix{Int64}:
140  97  74 160 131
 97 186  89 131  36
 74  89 152 144  71
160 131 144  54 142
131  36  71 142  36
```

Рис. 14: Приведение матриц к диагональному виду и вычисление

7. Найдите собственные значения матрицы A . Создайте диагональную матрицу из собственных значений матрицы A . Создайте нижнедиагональную матрицу из матрица A . Оцените эффективность выполняемых операций.

Задания для самостоятельного выполнения

```
8]: @time G = eigen(Q)
0.000082 seconds (19 allocations: 3.047 KiB)
8]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
values:
5-element Vector{Float64}:
-128.4932276488214
-55.887784553857845
42.75216727931888
87.16111477514491
542.4677381466137
vectors:
5x5 Matrix{Float64}:
-0.147575  0.647178  0.010882  0.548903  -0.507907
-0.256795  -0.173068  0.834628  -0.239064  -0.387253
-0.185537  0.239762  -0.422161  -0.731925  -0.440631
0.819784  -0.247506  -0.0273194  0.0366447  -0.514526
-0.453805  -0.657619  -0.352577  0.322668  -0.364928

9]: @time J = Diagonal(G.values)
0.000027 seconds (1 allocation: 16 bytes)
9]: 5x5 Diagonal{Float64, Vector{Float64}}:
-128.493      '      '      '      '
      '      -55.8878      '      '      '
      '      '      42.7522      '      '
      '      '      '      87.1611      '
      '      '      '      '      542.468

10]: # нижнетригональная матрица
@time T = LowerTriangular(Q)
0.000018 seconds (1 allocation: 16 bytes)
10]: 5x5 LowerTriangular{Int64, Matrix{Int64}}:
148      '      '      '      '
97 186      '      '      '
74 89 152      '      '
168 131 144 54      '
131 36 71 142 36
```

Рис. 15: Создание диагональной матрицы из собственных значений, нижнетригональной матрицы, оценка эффективности

Линейная модель экономики может быть записана как СЛАУ: $x - Ax = y$, где элементы матрицы A и столбца y — неотрицательные числа. По своему смыслу в экономике элементы матрицы A и столбцов x, y не могут быть отрицательными числами.

Задания для самостоятельного выполнения

```
1.51  2b  f1  242  2b

21): # 4.4.4. Лекция модели экономики

# Функция для проверки продуктивности через критерий неотрицательности
function is_productive_inverse(A)
    n = size(A, 1)
    E = Matrix{Float64}(I, n, n)
    try
        inv_matrix = inv(E - A)
        return all(inv_matrix .>= 0)
    catch
        return false # Если матрица (E-A) вырождена
    end
end

21): is_productive_inverse (generic function with 1 method)

22): # Функция для проверки продуктивности через спектральный критерий
function is_productive_spectral(A)
    λ = eigen(A).values
    return all(abs.(λ) .< 1)
end

22): is_productive_spectral (generic function with 1 method)

29): # Функция для проверки всех критериев
function check_productivity(A, name)
    println("Проверка матрицы $name:")
    println("A = ")
    println(A)

    inverse_productive = is_productive_inverse(A)
    spectral_productive = is_productive_spectral(A)

    println("Критерий (E-A)^-1 >= 0: $inverse_productive")
    println("Спектральный критерий |lambda| < 1: $spectral_productive")

    if inverse_productive == spectral_productive
        println("Оба критерия совпадают: матрица $inverse_productive ? \"продуктивна\" : \"не продуктивна\"")
    else
        println("Критерии не совпадают!")
    end
    println()
end

29): check_productivity (generic function with 1 method)
```

Рис. 16: Создание функций

Задания для самостоятельного выполнения

8. Матрица A называется продуктивной, если решение x системы при любой неотрицательной правой части y имеет только неотрицательные элементы x_i . Используя это определение, проверьте, являются ли матрицы продуктивными.
9. Критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все элементы матрица: $(E - A)^{-1}$ являются неотрицательными числами. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

Задания для самостоятельного выполнения

```
10: # 2
A1 = [1 2; 3 4]
A2 = 0.5 * [1 2; 3 4]
A3 = 0.1 * [1 2; 3 4]

check_productivity(A1, "A1 (исходная)")
check_productivity(A2, "A2 (<0.5)")
check_productivity(A3, "A3 (<0.1)")

Проверка матрицы A1 (исходная):
A =
[1 2; 3 4]
Критерий (E-A)^-1 == 0: false
Спектральный критерий (lambda) < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы A2 (<0.5):
A =
[0.5 1.0; 1.5 2.0]
Критерий (E-A)^-1 == 0: false
Спектральный критерий (lambda) < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы A3 (<0.1):
A =
[0.1 0.2; 0.3 0.4]
Критерий (E-A)^-1 == 0: true
Спектральный критерий (lambda) < 1: true
Оба критерия согласуются: матрица продуктивна

11: # 2
B1 = [1 2; 3 1]
B2 = 0.5 * [1 2; 3 1]
B3 = 0.1 * [1 2; 3 1]

check_productivity(B1, "B1 (исходная)")
check_productivity(B2, "B2 (<0.5)")
check_productivity(B3, "B3 (<0.1)")

Проверка матрицы B1 (исходная):
B =
[1 2; 3 1]
Критерий (E-B)^-1 == 0: false
Спектральный критерий (lambda) < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы B2 (<0.5):
B =
[0.5 1.0; 1.5 0.5]
Критерий (E-B)^-1 == 0: false
Спектральный критерий (lambda) < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы B3 (<0.1):
B =
[0.1 0.2; 0.3 0.1]
Критерий (E-B)^-1 == 0: true
Спектральный критерий (lambda) < 1: true
Оба критерия согласуются: матрица продуктивна

12: # По две матрицы B1, B2, B3 плюс вывод
end
```

Рис. 17: Проверка с помощью определения и критерия на продуктивность

10. Спектральный критерий продуктивности: матрица A является продуктивной тогда и только тогда, когда все её собственные значения по модулю меньше 1. Используя этот критерий, проверьте, являются ли матрицы продуктивными.

Задания для самостоятельного выполнения

```
# Те же матрицы B1, B2, B3 плюс новая
C1 = [1 2; 3 1]
C2 = 0.5 * [1 2; 3 1]
C3 = 0.1 * [1 2; 3 1]
C4 = [0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]

check_productivity(C1, "C1 (исходная)")
check_productivity(C2, "C2 (×0.5)")
check_productivity(C3, "C3 (×0.1)")
check_productivity(C4, "C4 (3×3 матрица)")

Проверка матрицы C1 (исходная):
A =
[1 2; 3 1]
Критерий (E-A)^-1 >= 0: false
Спектральный критерий |lambda| < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы C2 (×0.5):
A =
[0.5 1.0; 1.5 0.5]
Критерий (E-A)^-1 >= 0: false
Спектральный критерий |lambda| < 1: false
Оба критерия согласуются: матрица не продуктивна

Проверка матрицы C3 (×0.1):
A =
[0.1 0.2; 0.30000000000000004 0.1]
Критерий (E-A)^-1 >= 0: true
Спектральный критерий |lambda| < 1: true
Оба критерия согласуются: матрица продуктивна

Проверка матрицы C4 (3×3 матрица):
A =
[0.1 0.2 0.3; 0 0.1 0.2; 0 0.1 0.3]
Критерий (E-A)^-1 >= 0: true
Спектральный критерий |lambda| < 1: true
Оба критерия согласуются: матрица продуктивна
```

Рис. 18: Проверка с помощью спектрального критерия

Выводы

В результате выполнения данной лабораторной работы я изучила возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.